Unit 5

Exception Handling:

Introduction

Exception Handling in Java

The **Exception Handling in Java** is one of the powerful *mechanisms to handle the runtime errors* so that the normal flow of the application can be maintained.

In this tutorial, we will learn about Java exceptions, its types, and the difference between checked and unchecked exceptions.

What is Exception in Java?

Dictionary Meaning: Exception is an abnormal condition.

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

Advantage of Exception Handling

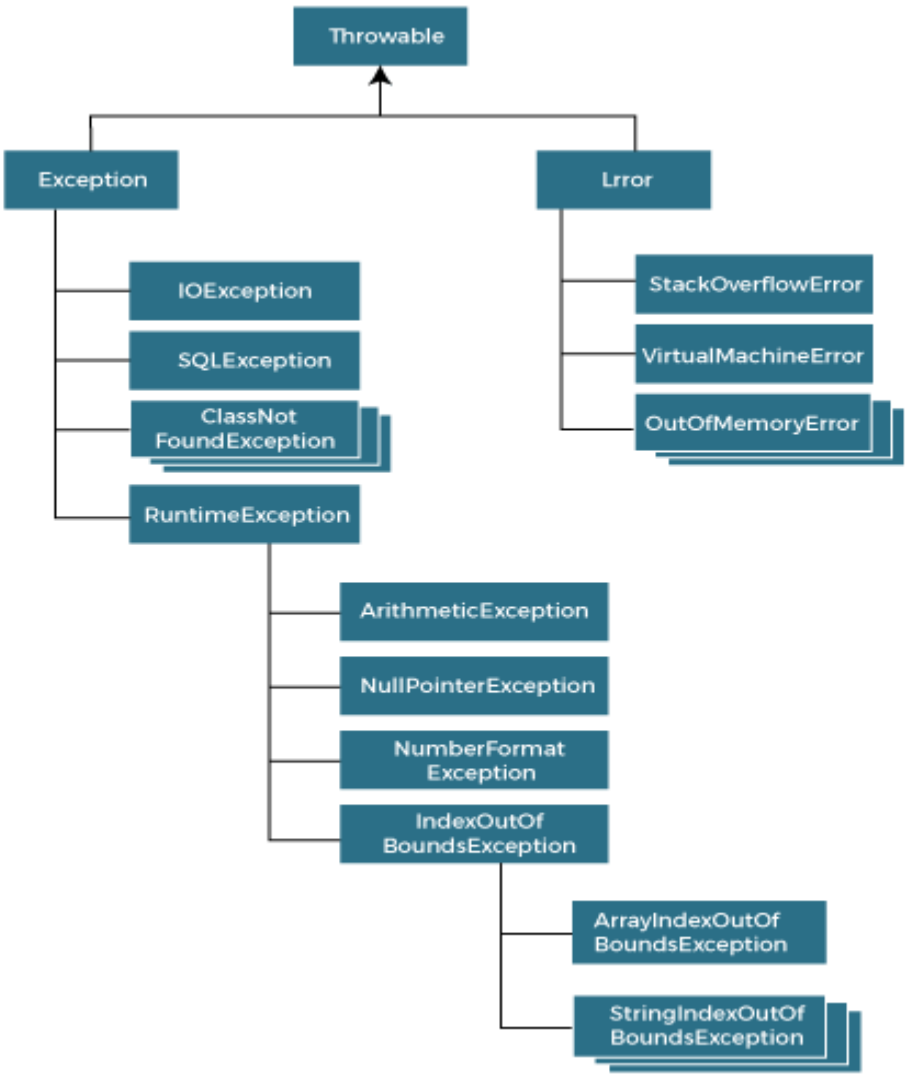
The core advantage of exception handling is **to maintain the normal flow of the application**. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

1. statement 1;
2. statement 2;
3. statement 3;
4. statement 4;
5. statement 5;//exception occurs
6. statement 6;

there are statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in Java.

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
Try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
Catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
Finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
Throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling Example

Let's see an example of Java Exception Handling in which we are using a try-catch statement to handle the exception.

JavaExceptionExample.java

```
public class JavaExceptionExample{
```

```
  public static void main(String args[]){
   try{
     //code that may raise exception
     int data=100/0;
   }catch(ArithmeticException e){
System.out.println(e);
}
   //rest code of the program
   System.out.println("rest of the code...");
  }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero

rest of the ……. code...

In the above example, 100/0 raises an ArithmeticException which is handled by a try-catch block.

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
1. int a=50/0;//ArithmeticException
```

2) A scenario where NullPointerException occurs

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

```
1. String s=null;
2. System.out.println(s.length());//NullPointerException
```

3) A scenario where NumberFormatException occurs

If the formatting of any variable or number is mismatched, it may result into NumberFormatException. Suppose we have a string variable that has characters; converting this variable into digit will cause NumberFormatException.

```
1. String s="abc";
2. int i=Integer.parseInt(s);//NumberFormatException
```

built in classes for Exception Handling

Java - Built-in Exceptions

Java defines several other types of exceptions that relate to its various class libraries. Following is the list of Java Unchecked RuntimeException.

Sr.No.	Exception & Description
1	**ArithmeticException** Arithmetic error, such as divide-by-zero.
2	**ArrayIndexOutOfBoundsException** Array index is out-of-bounds.
3	**ArrayStoreException** Assignment to an array element of an incompatible type.
4	**ClassCastException** Invalid cast.
5	**IllegalArgumentException** Illegal argument used to invoke a method.
6	**IllegalMonitorStateException** Illegal monitor operation, such as waiting on an unlocked thread.
7	**IllegalStateException** Environment or application is in incorrect state.
8	**IllegalThreadStateException** Requested operation not compatible with the current thread state.

9	**IndexOutOfBoundsException** Some type of index is out-of-bounds.
10	**NegativeArraySizeException** Array created with a negative size.
11	**NullPointerException** Invalid use of a null reference.
12	**NumberFormatException** Invalid conversion of a string to a numeric format.
13	**SecurityException** Attempt to violate security.
14	**StringIndexOutOfBounds** Attempt to index outside the bounds of a string.
15	**UnsupportedOperationException** An unsupported operation was encountered.

Mechanism of Exception Handling in Java

The mechanism of exception handling in Java is a systematic process used to **detect, throw, and handle runtime errors** so that the **normal flow of the program is maintained**. Java uses a set of predefined keywords to manage exceptions in an organized manner.

Keywords Used in Exception Handling

- **try** – encloses code that may cause an exception
- **catch** – handles the exception
- **finally** – executes regardless of exception occurrence
- **throw** – explicitly throws an exception
- **throws** – declares exceptions to be handled by the caller

Working Mechanism

1. The code that may generate an exception is placed inside the **try block**.
2. When an exception occurs, the JVM **creates an exception object**.

3. The JVM searches for a matching **catch block** to handle the exception.
4. If a suitable catch block is found, the exception is handled and program continues.
5. The **finally block** executes whether an exception occurs or not.

Syntax

```
try {
   // risky code
}
catch (ExceptionType e) {
   // exception handling code
}
finally {
   // cleanup code
}
```

Example Code

```
public class ExceptionTest {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
      } catch (ArithmeticException e) {
         System.out.println("Arithmetic Exception handled");
      } finally {
         System.out.println("Program continues");
      }
   }
}
```

Output

```
Arithmetic Exception handled
Program continues
```

- Division by zero causes an ArithmeticException.
- The exception is caught and handled using the catch block.
- The finally block executes irrespective of the exception.